```
8)      ts.add("F");
9)      ts.add("B");
10)     ts.add("E");
11)     ts.add("C");
12)     ts.add("A");
13)     System.out.println(ts);
14)     System.out.println(ts.ceiling("D"));
15)     System.out.println(ts.higher("D"));
16)     System.out.println(ts.floor("D"));
17)     System.out.println(ts.lower("D"));
18)     System.out.println(ts.descendingSet());
19)     ts.pollFirst();
20)     ts.pollLast();
21)     System.out.println(ts);
22)   }
23) }
```

TreeSet:

- It was introduced in JDK 1.2 version.
- It is not Legacy Collection.
- It has provided implementation for Collection, Set, SortedSet and NavigableSet interfaces.
- It is not index based.
- It is not allowing duplicate elements.
- It is not following insertion order.
- It follows Sorting order.
- It allows only homogeneous elements.
- It will not allow heterogeneous elements, if we are trying to add heterogeneous elements then JVM will rise an exception like java.lang.ClasscastException.
- It will not allow null values, if we are trying to add any null value then JVM will rise an exception like java.lang.NullPointerException.
- It able to allow only Comparable objects by default, if we are trying to add non comparable objects then JVM will rise an exception like java.lang.ClassCastException.

NOTE: If we are trying to add non comparable objects then we have to use java.util.Comparator.

- Its internal data structure is "Balanced Tree".
- It is mainly for frequent search operations.

Constructors:

- **public TreeSet()**
  It can be used to create an Empty TreeSet object.
  EX: TreeSet ts = new TreeSet();

- **public TreeSet(Comparator c)**
  It will create an empty TreeSet object with the explicit Sorting mechanism in the form of Comparator
  EX: TreeSet ts = new TreeSet(new MyComparator());

- **public TreeSet(SortedSet ts)**
  It will create TreeSet object with all elements of the specified SortedSet.

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        TreeSet ts1=new TreeSet();
7)        ts1.add("B");
8)        ts1.add("C");
9)        ts1.add("F");
10)       ts1.add("A");
11)       ts1.add("E");
12)       ts1.add("D");
13)       System.out.println(ts1);
14)       TreeSet ts2=new TreeSet(ts1);
15)       System.out.println(ts2);
16)    }
17) }
```

4) public TreeSet(Collection c)

It able to create TreeSet object with all the elements of the specified Collection.

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        ArrayList al=new ArrayList();
7)        al.add("B");
8)        al.add("C");
9)        al.add("F");
10)       al.add("A");
11)       al.add("E");
12)       al.add("D");
13)       System.out.println(al);
14)       TreeSet ts=new TreeSet(al);
15)       System.out.println(ts);
16)    }
17) }
```

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        TreeSet ts=new TreeSet();
7)        ts.add("B");
8)        ts.add("C");
9)        ts.add("F");
10)       ts.add("A");
11)       ts.add("E");
12)       ts.add("D");
13)       System.out.println(ts);
14)       ts.add("B");
15)       System.out.println(ts);
16)       //ts.add(null);--> NullPointerException
17)       //ts.add(new Integer(10));-->ClassCastException
18)       //ts.add(new StringBuffer("BBB"));->ClassCastException
19)    }
20) }
```

When we add elements to the TreeSet object, TreeSet object will arrange all the elements in a particular sorting order with the following algorithm.

- **TreeSet will construct a Tree [Balanced Tree] on the basis of the elements.**

  To construct Balanced Tree we have to use the following steps.
  - **If the element is first element to the TreeSet object then make that element as "Root Node".**
  - **If the element is not first element then access compareTo(--) method over the present element by passing previous elements one by one of the balanced Tree right from root node until the present element is located in Tree.**
    - **If compareTo(-) method returns -ve value then go to left child of the present node and access again compareTo(-) method by passing left child. If no left child is existed then make the present element as left child**
    - **If compareTo(-) method returns +ve value then go to right child and access again compareTo(-) by passing right as parameter. If no right child is existed then make the present element as right child.**
    - **If compareTo(-) method return 0 value then discard the present element and declare that the present element is a duplicate element of the existed element.**

- **TreeSet will retrieve all the elements from balanced Tree by following in order traversal.**
  In String class, compareTo(-) method was implemented like below
  str1.compareTo(str2);

  - **If str1 come first when compared with str2 as per dictionary order then compareTo() method will return -ve value.**
  - **If str2 come first when compared with str1 in dictionary order then compareTo() method will return +ve value.**
  - **If str1 and str2 are same or available at same location in dictionary order then compareTo(-) method will return 0 value.**

If we want to add user defined elements like Employee, Student, Customer to TreeSet then we have to use the following steps.
- **Declare an user defined class.**
- **Implement java.lang.Comparable iterface in User defined class.**
- **Provide implementation for compareTo(-) method in user defined class.**
- **In main class, in main() method, create objects for user defined class and add objects to TreeSet object.**

```
1)  package com.durgasoft.app05.test;
2)
3)  import java.util.TreeSet;
4)
5)  class Employee implements Comparable{
6)     int eno;
7)     String ename;
8)     float esal;
9)     String eaddr;
10)
11)    public Employee(int eno, String ename, float esal, String eaddr){
12)       this.eno = eno;
13)       this.ename = ename;
14)       this.esal = esal;
15)       this.eaddr = eaddr;
16)    }
17)
18)    @Override
19)    public int compareTo(Object obj) {
20)       Employee emp = (Employee)obj;
21)       int val = 0;
22)       val = this.ename.compareTo(emp.ename);
23)       return -val;
24)    }
25)
26)    @Override
27)    public String toString() {
28)       return "\nEmployee{" +
29)             "eno=" + eno +
30)             ", ename='" + ename + '\'' +
31)             ", esal=" + esal +
32)             ", eaddr='" + eaddr + '\'' +
33)             '}';
34)    }
35) }
36) public class Test {
37)    public static void main(String[] args) {
38)       Employee emp1 = new Employee(111, "Durga", 5000, "Hyd");
39)       Employee emp2 = new Employee(222, "Neelesh", 6000, "Hyd");
40)       Employee emp3 = new Employee(333, "Aishwary", 7000, "Hyd");
41)       Employee emp4 = new Employee(444, "Gopi Krishna", 8000, "Hyd");
42)       Employee emp5 = new Employee(555, "Tahauddin", 9000, "Hyd");
43)
44)       TreeSet ts = new TreeSet();
45)       ts.add(emp1);
```

```
46)       ts.add(emp2);
47)       ts.add(emp3);
48)       ts.add(emp4);
49)       ts.add(emp5);
50)
51)       System.out.println(ts);
52)    }
53) }
```

If we add non-comparable objects to TreeSet object then JVM will rise an exception like java.lang.ClassCastException, because, Non-Comparable objects are not providing compareTo(-) method internally, but, it is required to the TreeSet inorder to provide sorting order over elements.

If we want to add non-Comparable objects to TreeSet object then we must provide sorting logic to TreeSet object explicitly, for this, we have to use java.util.Comparator interface.

If we want to use Comparator interface in java applications then we have to use the following steps.

- **Declare an User defined class.**
- **Implement java.util.Comparator interface in user defined class.**
- **Provide implementation for Comparator interface methods in user defined class.**
  public boolean equals(Object obj)
  public int compare(Object obj1, Object obj2)

Note: In User defined class it is not required to implement equals(-) method, because, equals(-) method will come from default super class Object.

- **Provide User defined Comparator object to TreeSet object**
  EX: MyComparator mc = new MyComparator();
  TreeSet ts = new TreeSet(mc);

EX:

```
1)  import java.util.*;
2)  class MyComparator implements Comparator
3)  {
4)     public int compare(Object obj1, Object obj2)
5)     {
6)        StringBuffer s1=(StringBuffer)obj1;
7)        StringBuffer s2=(StringBuffer)obj2;
8)
9)        int length1=s1.length();
10)       int length2=s2.length();
11)       int val=0;
```

```
12)       if(length1<length2)
13)       {
14)          val=-100;
15)       }
16)       else if(length1>length2)
17)       {
18)          val=100;
19)       }
20)       else
21)       {
22)          val=0;
23)       }
24)       return -val;
25)    }
26) }
27) class Test
28) {
29)    public static void main(String[] args)
30)    {
31)       StringBuffer sb1=new StringBuffer("AAA");
32)       StringBuffer sb2=new StringBuffer("BB");
33)       StringBuffer sb3=new StringBuffer("CCCC");
34)       StringBuffer sb4=new StringBuffer("D");
35)       StringBuffer sb5=new StringBuffer("EEEEE");
36)       MyComparator mc=new MyComparator();
37)       TreeSet ts=new TreeSet(mc);
38)       ts.add(sb1);
39)       ts.add(sb2);
40)       ts.add(sb3);
41)       ts.add(sb4);
42)       ts.add(sb5);
43)       System.out.println(ts);
44)    }
45) }
```

EX:

```
1)  import java.util.*;
2)  class MyComparator implements Comparator
3)  {
4)     public int compare(Object obj1, Object obj2)
5)     {
6)        Student s1=(Student)obj1;
7)        Student s2=(Student)obj2;
8)
9)        int val=s1.saddr.compareTo(s2.saddr);
```

```
10)       return -val;
11)    }
12) }
13) class Student
14) {
15)    String sid;
16)    String sname;
17)    String saddr;
18)
19)    Student(String sid, String sname, String saddr)
20)    {
21)       this.sid=sid;
22)       this.sname=sname;
23)       this.saddr=saddr;
24)    }
25)    public String toString()
26)    {
27)       return "["+sid+","+sname+","+saddr+"]";
28)    }
29) }
30) class Test
31) {
32)    public static void main(String[] args)
33)    {
34)       Student std1=new Student("S-111", "Durga", "Hyd");
35)       Student std2=new Student("S-222", "Anil", "Chennai");
36)       Student std3=new Student("S-333", "Rahul", "Banglore");
37)       Student std4=new Student("S-444", "Rameshh", "Pune");
38)       MyComparator mc=new MyComparator();
39)       TreeSet ts=new TreeSet(mc);
40)       ts.add(std1);
41)       ts.add(std2);
42)       ts.add(std3);
43)       ts.add(std4);
44)       System.out.println(ts);
45)    }
46) }
```

Q) In Java applications, if we provide both implicit Sorting through Comparable and explicit sorting through Comparator to the TreeSet object at a time then which Sorting logic would be preferred by TreeSet inorder to Sort elements?

If we provide both implicit Sorting through Comparable and Explicit Sorting through Comparator to the TreeSet object at a time then TreeSet will take explicit Sorting through Comparator to sort all the elements.

EX:

```
1)  import java.util.*;
2)  class MyComparator implements Comparator
3)  {
4)     public int compare(Object obj1, Object obj2)
5)     {
6)        Customer cust1=(Customer)obj1;
7)        Customer cust2=(Customer)obj2;
8)
9)        int val=cust1.caddr.compareTo(cust2.caddr);
10)       return -val;
11)    }
12) }
13) class Customer implements Comparable
14) {
15)    String cid;
16)    String cname;
17)    String caddr;
18)
19)    Customer(String cid, String cname, String caddr)
20)    {
21)       this.cid=cid;
22)       this.cname=cname;
23)       this.caddr=caddr;
24)    }
25)    public String toString()
26)    {
27)       return "["+cid+","+cname+","+caddr+"]";
28)    }
29)    public int compareTo(Object obj)
30)    {
31)       Customer cust=(Customer)obj;
32)       int val=this.caddr.compareTo(cust.caddr);
33)       return val;
34)    }
35) }
36) class Test
37) {
38)    public static void main(String[] args)
39)    {
40)       Customer c1=new Customer("C-111", "Durga", "Hyd");
41)       Customer c2=new Customer("C-222", "Anil", "Chennai");
42)       Customer c3=new Customer("C-333", "Rahul", "Banglore");
43)       Customer c4=new Customer("C-444", "Ramesh", "Pune");
44)       MyComparator mc=new MyComparator();
```

```
45)       TreeSet ts=new TreeSet(mc);
46)       ts.add(c1);
47)       ts.add(c2);
48)       ts.add(c3);
49)       ts.add(c4);
50)       System.out.println(ts);
51)    }
52) }
```

Queue:

- It was introduced in JDK 5.0 version.
- It is a direct child interface to Collection Interface.
- It able to arrange all the elements as per FIFO [First In First Out], but, it is possible to change this algorithm as per our requirement.
- It able to allow duplicate elements.
- It is not following Insertion order.
- It is following Sorting order.
- It will not allow null values, if we add null value then JVM will rise an Exception like java.lang.NullPointerException.
- It will not allow heterogeneous elements, if we add heterogeneous elements then JVM will rise an exception like java.lang.ClassCastException.
- It able to allow only homogeneous elements.
- It able to allow comparable objects bydefault, if we add non comparable objects then JVM will rise an exception like java.lang.ClassCastException.
- If we want to add non comparable objects then we have to use Comparator.
- It able to manage all elements prior to process.

Methods:

- ## public void offer(Object obj)
  It can be used to insert the specified element to Queue.

- ## public Object peek()
  It can be used to return head element of the Queue.

- ## public Object element()
  It can be used to return head element of the Queue

Note: If we access peek() method on an empty Queue then peek() will return "null" value. If we access element() method on an empty Queue then element() method will rise an exception like java.util.NoSuchElementException

- **public Object poll()**
  It can be used to return and remove head element from Queue.

- **public Object remove()**
  It can be used to return and remove head element from Queue.

Note: If we access poll() method on an empty Queue then poll() method will return "null" value. If we access remove() method on an empty Queue then remove() method will rise an exception like "java.util.NoSuchElementException".

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        PriorityQueue q=new PriorityQueue();
7)        q.offer("A");
8)        q.offer("B");
9)        q.offer("C");
10)       q.offer("D");
11)       q.offer("E");
12)       q.offer("F");
13)       System.out.println(q);
14)       System.out.println(q.peek());
15)       System.out.println(q);
16)       System.out.println(q.element());
17)       System.out.println(q);
18)       /*
19)       PriorityQueue q1=new PriorityQueue();
20)       System.out.println(q1.peek());--> Null
21)       System.out.println(q1.element());--> Exception
22)       */
23)       System.out.println(q.poll());
24)       System.out.println(q);
25)       System.out.println(q.remove());
26)       System.out.println(q);
27)       /*
28)       PriorityQueue q1=new PriorityQueue();
29)       System.out.println(q1.poll());--> Null
30)       System.out.println(q1.remove());-->Exception
31)       */
32)    }
33) }
```

PriorityQueue:

- It was introduced in JDK 5.0 version.
- It is not Legacy Collection.
- It is a direct implementation class to Queue interface.
- It able to arrange all the elements prior to processing on the basis of the priorities.
- It able to allow duplicate elements.
- It is not following Insertion order.
- It is following Sorting order.
- It will not allow null values, if we add null value then JVM will rise an Exception like java.lang.NullPointerException.
- It will not allow heterogeneous elements, if we add heterogeneous elements then JVM will rise an exception like java.lang.ClassCastException.
- It able to allow only homogeneous elements.
- It able to allow comparable objects by default, if we add non comparable objects then JVM will rise an exception like java.lang.ClassCastException.
- If we want to add non comparable objects then we have to use Comparator.
- Its initial capacity 11 elements.
- It is not synchronized .
- No method is synchronized in PriorityQueue.
- It allows more than one thread at a time to access data.
- It follows parallel execution.
- It able to reduce application execution time.
- It able to improve application performance.
- It is not giving guarantee for Data consistency.
- It is not threadsafe.

Constructors:

- **public PriorityQueue()**

  It able to create an empty PriorityQueue object

  EX: PriorityQueue p = new PriorityQueue();

- **public PriorityQueue(int capacity)**

  It can be used to create an empty Queue with the specified capacity.

  EX: PriorityQueue p = new PriorityQueue(20);

- **public PriorityQueue(int capacity,Comparator c)**

  It able to create an empty PriorityQueue with explicit sorting logic throug COmparator and the specified capacity.

  EX: MyComparator mc = new MyComparator();
  PriorityQueue p = new PriorityQueue(20,mc);

- **public PriorityQueue(SortedSet ss)**
  It able to create PriorityQueue object with all the elements of the specified SortedSet.

EX:

```
TreeSet ts=new TreeSet();
ts.add("B");
ts.add("E");
ts.add("C");
ts.add("A");
ts.add("D");
System.out.println(ts);
PriorityQueue p = new PriorityQueue(ts);
System.out.println(p);
```

- **public PriorityQueue(Collection c)**
  It able to create PriorityQueue object with all the elements of the specified Collection.

EX:

```
ArrayList al = new ArrayList();
al.add("A");
al.add("B");
al.add("C");
al.add("D");
System.out.println(al);
PriorityQueue p = new PriorityQueue(al);
System.out.println(p);
```

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        PriorityQueue p=new PriorityQueue();
7)        p.add("A");
8)        p.add("D");
9)        p.add("B");
10)       p.add("C");
11)       p.add("F");
12)       p.add("E");
13)       System.out.println(p);
14)       p.add("B");
15)       System.out.println(p);
16)       //p.add(null);-->NullPointerException
17)       //p.add(new Integer(10));->ClassCastException
```

```
18)       //p.add(new StringBuffer("BBB"));->ClassCastException
19)    }
20) }
```

Map:

- It was introduced in JDK 1.2 version.
- It is not child interface to Collection Interface.
- It able to arrange all the elements in the form of Key-value pairs.
- In Map, both keys and values are objects.
- Duplicates are not allowed at keys, but values may be duplicated.
- Only one null value is allowed at keys side, but, any number of null values are allowed at values side.
- Both keys and Values are able to allow heterogeneous elements.
- Insertion order is not followed.
- Sorting order is not followed.

Methods:

- **public void put(Object key, Object value)**
  It will add the specified key-value pair to Map.

- **public void putAll(Map m)**
  It will add all key-value pairs of the specified map to the present Map object.

- **public Object get(Object key)**
  It will return value of the specified key.

- **public Object remove(Object key)**
  It will remove a key-value pair from Map on the basis of the specified key.

- **public int size()**
  It will return number of key-value pairs of a Map

- **public boolean containsKey(Object key)**
  It will check whether the specified key is existed or not at keys side.

- **public boolean cotainsValue(Object key)**
  It will check whether the specified value is available or not at values side.

- **public Set keySet()**
  It will return all keys in the form of a Set.

- **public Collection values()**
  It will return all values in the form of a Collection object.

- **public boolean isEmpty()**
  It will check whether the Map object is empty or not, if the present map object is empty then it will return true value otherwise it will return false value.

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        HashMap hm=new HashMap();
7)        hm.put("A","AAA");
8)        hm.put("B","BBB");
9)        hm.put("C","CCC");
10)       hm.put("D","DDD");
11)       hm.put("E","EEE");
12)       System.out.println(hm);
13)       HashMap hm1=new HashMap();
14)       hm1.put("X","XXX");
15)       hm1.put("Y","YYY");
16)       System.out.println(hm1);
17)       hm.putAll(hm1);
18)       System.out.println(hm);
19)       System.out.println(hm.get("B"));
20)       System.out.println(hm.remove("E"));
21)       System.out.println(hm.size());
22)       System.out.println(hm.isEmpty());
23)       System.out.println(hm.containsKey("D"));
24)       System.out.println(hm.containsValue("DDD"));
25)       System.out.println(hm.keySet());
26)       System.out.println(hm.values());
27)    }
28) }
```

HashMap:

- It was introduced in JDK1.2 version.
- It is not Legacy
- It is an implementation class to Map interface.
- It able to arrange all the elements in the form of Key-value pairs.
- In HashMap, both keys and values are objects.
- Duplicates are not allowed at keys, but values may be duplicated.
- Only one null value is allowed at keys side, but, any number of null values are allowed at values side.
- Both keys and Values are able to allow heterogeneous elements.
- Insertion order is not followed.
- Sorting order is not followed.
- Its internal data structure is "Hashtable".
- Its initial capacity is 16 elements.
- It is not synchronized
- No method is synchronized in HashMap
- It allows more than one thread to access data.
- It follows parallel execution.
- It will reduce application execution time.
- It will improve application performance.
- It is not giving guarantee for data consistency.
- It is not threadsafe.

Constructors:

- public HashMap()
- public HashMap(int capacity)
- public HashMap(int capacity, float fill_Ratio)
- public HashMap(Map m)

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        HashMap hm=new HashMap();
7)        hm.put("A","AAA");
8)        hm.put("B","BBB");
9)        hm.put("C","CCC");
10)       hm.put("D","DDD");
11)       hm.put("E","EEE");
12)       System.out.println(hm);
13)       hm.put("B","FFF");
14)       System.out.println(hm);
```

```
15)       hm.put("F","CCC");
16)       System.out.println(hm);
17)       hm.put(null,"GGG");
18)       hm.put(null,"HHH");
19)       hm.put("G",null);
20)       hm.put("H",null);
21)       System.out.println(hm);
22)       hm.put(new Integer(10), new Integer(20));
23)       System.out.println(hm);
24)    }
25) }
```

LinkedHashMap:

Q) What are the differences between *HashMap* and *LinkedHashMap?*

- HashMap was introduced in JDK 1.2 version.
  LinkedHashMap was introduced in JDK 1.4 version.

- HashMap is not following insertion order.
  LinkedHashMap is following insertion order.

- HashMap internal data structure is Hashtable.
  LinkedHashMap internal data structure is Hashtable+LinkedList

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        HashMap hm=new HashMap();
7)        hm.put("A","AAA");
8)        hm.put("B","BBB");
9)        hm.put("C","CCC");
10)       hm.put("D","DDD");
11)       hm.put("E","EEE");
12)       System.out.println(hm);
13)
14)       LinkedHashMap lhm=new LinkedHashMap();
15)       lhm.put("A","AAA");
16)       lhm.put("B","BBB");
17)       lhm.put("C","CCC");
```

```
18)      lhm.put("D","DDD");
19)      lhm.put("E","EEE");
20)      System.out.println(lhm);
21)   }
22) }
```

IdentityHashMap:

Q) What are the differences between *HashMap* and *IdentityHashMap?*

- HashMap was introduced in JDK 1.2 version.
  IdentityHashMap was introduced in JDK 1.4 version.

- HashMap and IdentityhashMap are not allowing duplicate keys, to check duplicate keys HashMap will use equals(-) method, but, IdentityHashMap will use '==' operator.

Note: '==' operator will perform references comparison always, but, equals() method was defined in Object class initially, later on it was overridden in String class and in all wrapper classes in order to perform contents comparison.

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        Integer in1=new Integer(10);
7)        Integer in2=new Integer(10);
8)        HashMap hm=new HashMap();
9)        hm.put(in1,"AAA");
10)       hm.put(in2,"BBB");// in2.equals(in1)-->true
11)       System.out.println(hm);// {10=BBB}
12)
13)       IdentityHashMap ihm=new IdentityHashMap();
14)       ihm.put(in1, "AAA");
15)       ihm.put(in2, "BBB");// in2 == in1 --> false
16)       System.out.println(ihm);// {10=AAA, 10=BBB}
17)    }
18) }
```

WeakHashMap:

Q) What is the difference between *HashMap* and *WeakHashMap?*

- Once if we add an element to HashMap then HashMap is not allowing Garbage Collector to destroy its objects.
- Even if we add an element to WeakHashMap then WeakHashMap is able to allow Garbage Collector to destroy elements.

EX:

```
1)  import java.util.*;
2)  class A
3)  {
4)     public String toString()
5)     {
6)        return "A";
7)     }
8)  }
9)  class Test
10) {
11)    public static void main(String[] args)
12)    {
13)       A a1=new A();
14)       HashMap hm=new HashMap();
15)       hm.put(a1, "AAA");
16)       System.out.println("HM Before GC  :"+hm);// {A=AAA}
17)       a1=null;
18)       System.gc();
19)       System.out.println("HM After GC   :"+hm);// {A=AA}
20)
21)       A a2=new A();
22)       WeakHashMap whm=new WeakHashMap();
23)       whm.put(a2, "AAA");
24)       System.out.println("WHM Before GC :"+whm);// {A=AAA}
25)       a2=null;
26)       System.gc();
27)       System.out.println("WHM After GC  :"+whm);// {}
28)
29)    }
30) }
```

NOTE: In Java applications, Garbage Collector will destroy objects internally. In java applications, it is possible to destroy objects explicitly by activating GarbageCollector, for this, we have to use the following two steps.

- Nullify the respective object reference.
- Access System.gc() method, it will access finalize() method internally just before destroying object.

EX:

```
1)  class A {
2)      A() {
3)              System.out.println("Object Creating");
4)      }
5)      public void finalize() {
6)          System.out.println("Object Destroying");
7)      }
8)  }
9)  class Test {
10)     public static void main(String[] args) {
11)         A a = new A();
12)         a = null;
13)         System.gc();
14)     }
15) }
```

SortedMap:

- It was introduced in JDK1.2 version.
- It is a direct child interface to Map interface
- It able to allow elements in the form of Key-Value pairs, where both keys and values are objects.
- It will not allow duplicate elements at keys side, but, it able to allow duplicate elements at values side.
- It will not follow insertion order.
- It will follow sorting order.
- It will not allow null values at keys side. If we are trying to add null values at keys side then JVM will rise an exception like java.lang.NullPointerException.
- It will not allow heterogeneous elements at keys side, if we are trying add heterogeneous elements then JVM will rise an exception like java.lang.ClassCastException.
- It able to allow only comparable objects at keys side by default, if we are trying to add non comparable objects then JVM will rise an exception like java.lang.ClassCastException.
- If we want to add non comparable objects then we must use Comparator.

Methods:

public Object firstKey()
public Object lastKey()
public SortedMap headMap(Object key)
public SportedMap tailMap(Object key)
public SortedMap subMap(Object obj1, Object obj2)

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        TreeMap tm=new TreeMap();
7)        tm.put("B", "BBB");
8)        tm.put("E", "EEE");
9)        tm.put("D", "DDD");
10)       tm.put("A", "AAA");
11)       tm.put("F", "FFF");
12)       tm.put("C", "CCC");
13)       System.out.println(tm);
14)       System.out.println(tm.firstKey());
15)       System.out.println(tm.lastKey());
16)       System.out.println(tm.headMap("D"));
17)       System.out.println(tm.tailMap("D"));
18)      System.out.println(tm.subMap("B", "E"));
19)   }
20) }
```

NavigableMap:

It was introduced in JAVA6 version, it is a child interface to SortedMap and it has defined methods to provide navigations over the elements.

Methods:

- **public Object CeilingKey(Object obj)**
- **public Object higherKey(Object obj)**
- **public Object floorKey(Object obj)**
- **public Object lowerKey(Object obj)**
- **public NavigableMap descendingMap()**
- **public Map.Entry pollFirstEntry()**
- **public Map.Entry pollLastEntry()**

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        TreeMap tm=new TreeMap();
7)        tm.put("A", "AAA");
8)        tm.put("B", "BBB");
9)        tm.put("C", "CCC");
10)       tm.put("D", "DDD");
11)       tm.put("E", "EEE");
12)       tm.put("F", "FFF");
13)       System.out.println(tm);
14)       System.out.println(tm.descendingMap());
15)       System.out.println(tm.ceilingKey("D"));
16)       System.out.println(tm.higherKey("D"));
17)       System.out.println(tm.floorKey("D"));
18)       System.out.println(tm.lowerKey("D"));
19)       System.out.println(tm.pollFirstEntry());
20)       System.out.println(tm.pollLastEntry());
21)       System.out.println(tm);
22)    }
23) }
```

TreeMap:

- It was introduced in JDK 1.2 version.
- It is not Legacy.
- It is an implementation class to Map, SoortedMap and NavigableMap interfaces.
- It able to allow elements in the form of Key-Value pairs, where both keys and values are objects.
- It will not allow duplicate elements at keys side, But, it able to allow duplicate elements at values side.
- It will not follow insertion order.
- It will follow sorting order.
- It will not allow null values at keys side. If we are trying to add null values at keys side then JVM will rise an exception like java.lang.NullPointerException.
- It will not allow heterogeneous elements at keys side, if we are trying add heterogeneous elements then JVM will rise an exception like java.lang.ClassCastException.
- It able to allow only comparable objects at keys side by default, if we are trying to add non comparable objects then JVM will rise an exception like java.lang.ClassCastException.
- If we want to add non comparable objects then we must use Comparator.

- Its internal data Structer is "Red-Black Tree".
- It is not synchronized.
- No methods are synchronized in TreeMap.
- It allows more than one thread to access data.
- It will follow parallel execution.
- It will reduce execution time.
- It will improve application performance.
- It is not giving guarantee for Data Consistency.
- It is not threadsafe.

Constructors:

- public TreeMap()
- public TreeMap(Comparator c)
- public TreeMap(SortedMap sm)
- public TreeMap(Map m)

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        TreeMap tm=new TreeMap();
7)        tm.put("A", "AAA");
8)        tm.put("B", "BBB");
9)        tm.put("C", "CCC");
10)       tm.put("D", "DDD");
11)       System.out.println(tm);
12)       tm.put("B", "EEE");
13)       System.out.println(tm);
14)       tm.put("E", "CCC");
15)       System.out.println(tm);
16)       //tm.put(null, "EEE");-->NullPointerException
17)       tm.put("F",null);
18)       System.out.println(tm);
19)       //tm.put(new Integer(10), new Integer(20));-->CCE
20)       System.out.println(tm);
21)       tm.put("G", new Integer(20));
22)       System.out.println(tm);
23)       //tm.put(new StringBuffer("BBB"), "GGG");-->CCE
24)       tm.put("H", new StringBuffer("HHH"));
25)       System.out.println(tm);
26)    }
27) }
```

Hashtable:

Q) What are the differences between *HashMap* and *Hashtable?*

- **HashMap was introduced in JDK 1.2 version.**
  Hashtable was introduced in JDK 1.0 version.

- **HashMap is not Legacy.**
  Hashtable is Legacy.

- **In HashMap, one null value is allowed at keys side and any number of null values are allowed at values side.**
  In case of Hashtable, null values are not allowed at both keys and values side.

- **HashMap is not synchronized.**
  Hashtable is synchronized.

- **No method is synchronized in HashMap.**
  Almost all the methods are synchronized in Hashtable

- **HashMap allows more than one thread to access data.**
  Hashtable allows only one thread at a time to access data.

- **HashMap follows parallel execution.**
  Hashtable follows sequential execution.

- **HashMap will reduce execution time.**
  Hashtable will increase execution time.

- **HashMap will improve application performance.**
  Hashtable will reduce application performance.

- **HashMap will not give guarantee for data consistency.**
  Hashtable will give guarantee for data consistency

- **HashMap is not threadsafe.**
  Hashtable is threadsafe.

EX:

```
1)  import java.util.*;
2)  class Test
3)  {
4)     public static void main(String[] args)
5)     {
6)        HashMap hm=new HashMap();
7)        hm.put("A", "AAA");
8)        hm.put("B", "BBB");
9)        hm.put("C", "CCC");
10)       hm.put("D", "DDD");
11)       System.out.println(hm);
12)       hm.put(null, "EEE");
13)       hm.put(null, "FFF");
14)       hm.put("E",null);
15)       hm.put("F", null);
16)       System.out.println(hm);
17)       System.out.println();
18)       Hashtable ht=new Hashtable();
19)       ht.put("A", "AAA");
20)       ht.put("B", "BBB");
21)       ht.put("C", "CCC");
22)       ht.put("D", "DDD");
23)       System.out.println(ht);
24)       //ht.put(null, "EEE");-->NullPointerException
25)       //ht.put("E", null);-->NullPointerException
26)    }
27) }
```

Properties:

In Java applications, if we have any data which we want to change frequently then we have to manage that type of data in a properties file, otherwise we have to perform recompilation on every modification.

The main purpose of properties files in java applications is,

- **To manage labels of the GUI components in GUI appl.**
- **To manage locale respective messages in I18N Appl.**
- **To manage exception messages in Exception handling.**
- **To manage validation messages in Data validations.**

EX:

user.properties

uname=User Name
upwd=User password
uname.required=User Name is Required
upwd.required=User Password is required
exception.insufficientfunds=Funds are not sufficient in your Account.

In Java applications, to represent data of a particular properties file we have to use java.util.Properties class.

To get data from a particular properties file to Properties object we have to use the following steps.

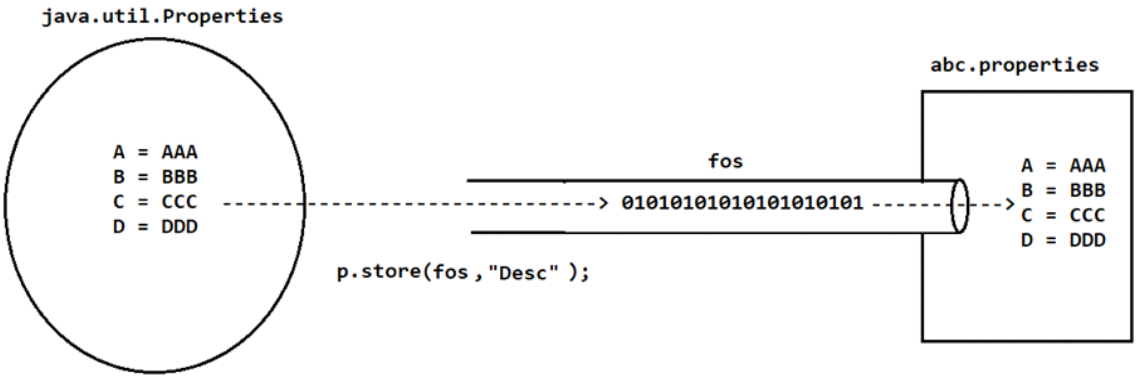
- **Create Properties file with the data in the form of Key-value pairs.**
- **Create Properties class object.**
- **Create FileInputStream to get data from properties file.**
- **Load data from FileInputStream to Properties object by using the following method.**
  public void load(FileInputStream fis)
- **Get data from Properties object by using the following method.**
  public String getProperty(String key)

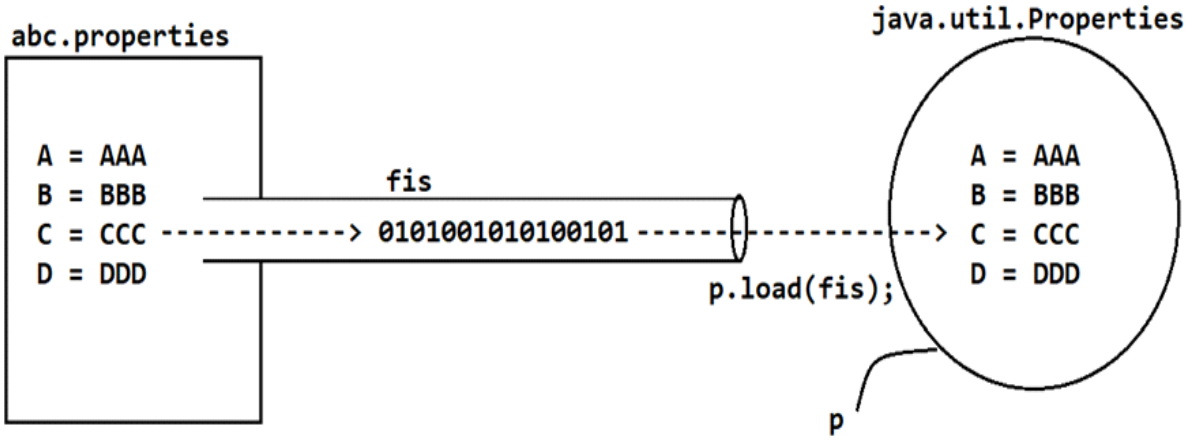
To send data from Properties object to properties file we have to use the following steps.

- **Create Properties class object.**
- **Set data to Properties object by using the following method.**
  public void setProperty(String key, String val)
- **Create FileOutputStream with the target file.**
- **Store Properties object data to FileOutputStream by using the following method.**
  public void store(FileOutputStream fos, String des)

db.properties

driver_Class=sun.jdbc.odbc.JdbcOdbcDriver
driver_URL=jdbc:odbc:dsn_name
db_User=system
db_Password=durga





Test.java

```
1)  import java.util.*;
2)  import java.io.*;
3)  class Test
4)  {
5)     public static void main(String[] args)throws Exception
6)     {
7)        Properties p=new Properties();
8)        FileInputStream fis=new FileInputStream("db.properties");
9)        p.load(fis);
10)       System.out.println("JDBC Parameters");
11)       System.out.println("--------------------");
12)       System.out.println("Driver_Class  :"+p.getProperty("driver_Class"));
13)       System.out.println("Driver_URL    :"+p.getProperty("driver_URL"));
14)       System.out.println("DB_User       :"+p.getProperty("db_User"));
15)       System.out.println("DB_PAssword   :"+p.getProperty("db_Password"));
16)    }
17) }
```

Test1.java

```
1)  import java.util.*;
2)  import java.io.*;
3)  class Test1
4)  {
5)     public static void main(String[] args)throws Exception
6)     {
7)        Properties p=new Properties();
8)        p.setProperty("uname","Durga");
9)        p.setProperty("upwd", "durga123");
10)       p.setProperty("uqual", "M Tech");
11)       p.setProperty("uemail", "durga@durgasoft.com");
12)       p.setProperty("umobile","91-9988776655");
13)
14)       FileOutputStream fos=new FileOutputStream("user.properties");
15)       p.store(fos,"User Details");
16)    }
17) }
```

Generics